

Motion Planning Diffusion: Learning and Planning of Robot Motions with Diffusion Models

João Carvalho¹, An T. Le¹, Mark Baierl¹, Dorothea Koert^{1,4} and Jan Peters^{1,2,3,4}

Abstract—Learning priors on trajectory distributions can help accelerate robot motion planning optimization. Given previously successful plans, learning trajectory generative models as priors for a new planning problem is highly desirable. Prior works propose several ways on utilizing this prior to bootstrapping the motion planning problem. Either sampling the prior for initializations or using the prior distribution in a maximum-a-posterior formulation for trajectory optimization. In this work, we propose learning diffusion models as priors. We then can sample directly from the posterior trajectory distribution conditioned on task goals, by leveraging the inverse denoising process of diffusion models. Furthermore, diffusion has been recently shown to effectively encode data multi-modality in high-dimensional settings, which is particularly well-suited for large trajectory dataset. To demonstrate our method efficacy, we compare our proposed method - Motion Planning Diffusion - against several baselines in simulated planar robot and 7-dof robot arm manipulator environments. To assess the generalization capabilities of our method, we test it in environments with previously unseen obstacles. Our experiments show that diffusion models are strong priors to encode high-dimensional trajectory distributions of robot motions. <https://sites.google.com/view/mp-diffusion>

I. INTRODUCTION

Motion planning is a crucial component of autonomous robot systems [1], [2], [3], [4]. It addresses the problem of finding a feasible, smooth, and collision-free path between a start and a goal point in a robot’s configuration space, which can subsequently be executed by a lower-level controller [5].

Commonly used approaches for motion planning are either sampling [6], [7], [8], [9] or optimization based [2], [10], [11], [3]. Sampling-based methods possess a *completeness* property, assuring a global optimum given infinite compute time [9]. However, in practice, they often suffer from sample inefficiency and tend to produce non-smooth trajectories [12]. On the other hand, optimization-based planners optimize initial trajectories via either preconditioned gradient descent [2], [10], [11]; or stochastic update rules [3], [13] and can integrate desired properties such as smoothness as costs to be optimized. Nevertheless, optimization-based planners depend on a good initialization and can get trapped in local minima due to the non-convexity of complex problems. Specifically, they commonly require a good initialization prior and well-tuned hyperparameters to work well [11], [13].

Recently, learning-based methods have shown promising potential to improve classical motion planning [14], e.g. by utilizing experience from previously successful plans [15],

This work was funded by the German Federal Ministry of Education and Research project IKIDA (01IS20045), and by the German Research Foundation project METRIC4IMITATION (PE 2315/11-1). ¹Intelligent Autonomous Systems Lab, TU Darmstadt, Germany; ²German Research Center for AI (DFKI); ³Hessian.AI; ⁴Centre for Cognitive Science

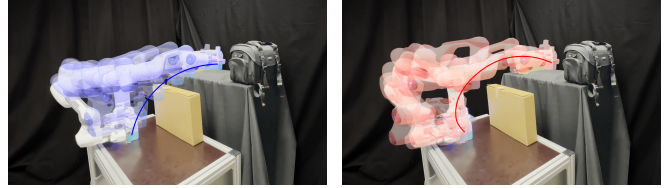


Fig. 1: Execution of motions in the real-world Panda Shelf environment. The motion in blue and red start and end at the same configurations, but it is possible to see two modes resulting from sampling trajectories with MPD. This environment includes obstacles (represented as boxes in the digital twin), which are not present in the environment used for training (Fig. 2d).

[16], [13] or incorporating priors from human demonstrations [17], [18], [19]. In particular, sampling from learned prior distributions, conditioned on contexts such as start/goal configurations and environmental variables, can provide good initializations for motion planners [13], [20], [21].

Following the imitation learning perspective [22], in this work, we learn the prior model from expert data and incorporate it into optimization-based motion planning. In particular, instead of explicitly sampling from a prior distribution as motion planning initialization, we propose to merge the prior sampling and motion optimization into one algorithm, by leveraging recent formulations in diffusion-based generative models [23], [24]. This type of *implicit* model has shown impressive results in modeling multimodal and high-dimensional data, such as image generation [25], [24], [26], [27], having superior generative performance (with/out context guidance) compared to previous generative models [28], [29]. Indeed, these diffusion model properties are particularly well-suited for learning from demonstrations in robotics manipulation settings, where state space dimensions are usually large in manipulators (e.g. full state of position and velocity of Franka Emika Panda arm has 14 dimensions) and there exist thousands of trajectory samples in the expert dataset. Furthermore, as shown later, the diffusion sampling process works well with gradients from standard motion planning costs, allowing for better and more diverse multimodal trajectory solutions.

Our main **contributions** are: (1) we learn a trajectory generative model with a diffusion model using expert trajectories generated with an optimal motion planning algorithm; (2) we formulate the motion planning problem as planning-as-inference by sampling from a posterior distribution leveraging guidance in diffusion models; (3) to validate our approach we present results in several environments with increasing difficulty; (4) we empirically demonstrate that learning and sampling from the diffusion model speeds up motion planning without an informed prior and is better than a commonly used generative model.

II. RELATED WORKS

There exists a huge body of literature on learning to plan for robotics. In this section, we focus on discussing related works that combine recent learning methods with classic motion planning approaches (sec. II-A). Additionally, we provide a short background on applications of diffusion models [25], [24] in robotics (sec. II-B).

A. Learning to plan for motion planning

We survey learning methods for both sampling and optimization-based planners. Addressing the sample-inefficiency of sampling-based planners such as Probabilistic Road Maps (PRM) [6] and Rapid Exploring Random Trees (RRT) [7], several works have proposed learning conditional sampling distributions using the environment and task information as context variables. For instance, [15] learns to generate collision-free samples using a Conditional Variational AutoEncoder (CVAE) conditioned on an occupancy map. Interestingly, [30] proposes Motion Planning Transformers, which determines informative regions for sampling new nodes for RRT-like methods. Other works [16], [31], [32] learn a conditioned one-step neural planner and promote diverse solutions by adding dropout ensuring stochasticity. In contrast to these methods, we learn a trajectory distribution model to easily introduce trajectory time-correlated constraints such as smoothness, which is important in robot motions.

Contrary to sampling-based planners, trajectory optimization methods directly optimize whole trajectories, aiming for smoothness while satisfying other objective constraints. However, they typically heavily depend on initialization and suffer from local minima, e.g. speeding up through obstacles, and thus learning better initial distributions can speed up and improve the success rate of these methods. Several works use Learning from Demonstration (LfD) (i.e., Behavioral Cloning) to encode trajectory priors [18], [17], [19]. They fit a Gaussian Mixture Model (GMM) given a set of demonstrations and then use these as priors for motion optimization. However, GMM fitting typically cannot capture well multi-modal trajectories in high-dimensional spaces. For better learning the multi-modality of expert trajectories, [13] learned Energy-Based Models (EBMs) that capture expert data objectives. They formulate the trajectory optimization problem via planning-as-inference to incorporate the EBM priors as planning costs. In contrast to incorporating prior as cost, we directly learn a trajectory generator using diffusion models to assist the optimization, where sampling high-dimensional trajectories are difficult for EBMs [33].

B. Diffusion models for robotics

Few works have explored using score-based and diffusion models in robotics. At task planning levels, [34] recently proposes DALL-E-BOT, which generates a text description from a scene image and then prompts the text-to-image generator DALL-E [28] for a “goal scene”, thus identifying the desired poses. [35] proposes *StructDiffusion* for arranging objects based on language commands, by predicting the object arrangements using a language-conditioned diffusion

model. On learning fine-grained trajectory structure, recently, diffusion models have been used to generate human-like motions [36]. In our previous work [37], we showed a simple example of how to use conditioned score-based models to generate trajectories for different environments. Mostly similar to our work, [20] used diffusion to learn cost functions for jointly optimizing motion and grasping poses in SE(3). [38] introduced the Diffuser, a trajectory generative model used for planning in Offline RL and long-horizon tasks. [39] proposes the Decision Diffuser learned over position-only trajectories that can be conditioned with classifier-free guidance skill generation or constraint satisfaction. In contrast to these works, we bring diffusion models closer to motion planners, by incorporating diffusion models as priors combined with differentiable cost likelihoods and also by learning higher trajectory derivatives. We show that, via our formulation, we can directly sample optimal trajectories from the posterior by following the reverse diffusion sampling process, contrasting to sampling from the optimized proposal distribution as in [13].

III. MOTION PLANNING DIFFUSION

In this section, we describe our method - Motion Planning Diffusion (MPD). First, we introduce the motion planning-as-inference perspective, laying the ground for incorporating diffusion into motion planning. We then explain how to learn trajectory distributions with diffusion models. Finally, with MPD, we show how to sample from the trajectory posterior via the reverse diffusion process while incorporating the cost likelihood representing other motion planning objectives.

A. Motion Planning

Let $\mathbf{s} = [\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top \subseteq \mathbb{R}^d$ encode the state of a robot, where $\mathbf{q}$ is the robot’s configuration position (e.g. the joint space), $\dot{\mathbf{q}}$ is the robot’s configuration velocity, d is state space dimension. Let $\mathcal{O}$ represent a task (or objective) the robot has to perform in its environment. A trajectory $\boldsymbol{\tau} \triangleq (\mathbf{s}_0, \dots, \mathbf{s}_{H-1}) \in \mathbb{R}^{H \times d}$ represented as waypoints is a sequence of states in discrete-time with horizon H . In this work, we consider only states and assume a controller brings the robot from state $\mathbf{s}_i$ to $\mathbf{s}_{i+1}$, e.g., using a PD or inverse dynamics controller.

In motion planning, $\mathcal{O}$ typically represents a task that encodes a collision-free path between start and goal states. This objective is commonly represented with a set of costs $c_i(\boldsymbol{\tau})$, which encode the robot’s start and goal configurations, the trajectory’s smoothness, and the collision-free constraint. Optimization-based motion planning formulates the motion planning problem as a trajectory optimization one [13]

$$\boldsymbol{\tau}^* = \arg \min_{\boldsymbol{\tau}} \sum_i \lambda_i c_i(\boldsymbol{\tau}), \quad (1)$$

where $\lambda_i > 0$ are different weights for the costs. Common approaches for solving this problem are either preconditioned gradient methods [2], [11], which rely on carefully designed first-order differentiable cost functions; or stochastic methods [3], [13], which evaluate the sampled trajectories and update the sampling distribution by weighing the particles on arbitrary non-smooth and non-convex costs.

B. Motion Planning as Inference

The connection between trajectory optimization and probabilistic inference is well established [40], [41], [42], [43], [44], [45]. In the planning-as-inference framework, the goal is to sample from the posterior distribution of trajectories given the task objective

$$p(\tau|\mathcal{O}) \propto p(\mathcal{O}|\tau)p(\tau),$$

where $p(\tau)$ can be interpreted as a prior over trajectories in the environment, and $p(\mathcal{O}|\tau)$ is the likelihood of achieving the task goals. A common assumption is that the likelihood factorizes into independent components [13]

$$p(\mathcal{O}|\tau) \propto \prod_i p_i(o_i|\tau)^{\lambda_i} \quad (2)$$

with $\lambda_i > 0$ as temperatures of planning objective distributions [46]. Assuming p_i belongs to the exponential family, we can arbitrarily write $p_i(o_i|\tau) \propto \exp(-c_i(\tau))$. Then, performing Maximum-a-Posteriori (MAP) on the trajectory posterior

$$\begin{aligned} \tau^* &= \arg \max_{\tau} \log p(\mathcal{O}|\tau)p(\tau) \\ &= \arg \max_{\tau} \log \prod_i p_i(o_i|\tau)^{\lambda_i} + \log p(\tau) \\ &= \arg \min_{\tau} \sum_i \lambda_i c_i(\tau) - \log p(\tau) \end{aligned} \quad (3)$$

is equivalent to the motion planning problem (1) minus the log of the prior. Contrary to classical optimization-based motion planning, formulating motion planning as an inference problem has several advantages. Notably, the inference framework provides a principal way to introduce informative priors to planning problems, e.g., Gaussian Process Motion Planning (GPMP) [47], [11] utilizes a Gaussian Process (GP) to encode dynamic feasibility and smoothness as trajectory priors. In this work, we leverage this advantage to incorporate the learned diffusion prior encoded the expert data.

C. Diffusion Models as Trajectory Generative Models

Let us consider the unconditional diffusion model on trajectories. Diffusion models transform a trajectory from the data distribution $\tau_0 \sim q(\tau_0)$ into white Gaussian noise by running a Markovian forward diffusion process $q(\tau_t|\tau_{t-1}, t) = \mathcal{N}(\tau_t; \sqrt{1-\beta_t}\tau_{t-1}, \beta_t\mathbf{I})$, where $t = 1, \dots, N$ is the diffusion time step (not the trajectory index), N is the number of diffusion steps, and β_t is the noise scale at time step t . Common schedules for β are linear, cosine or exponential [24], [48]. Assuming the data τ_0 lives in Euclidean space, the distribution of the diffusion process at time step t is Gaussian and can be written in closed-form as $q(\tau_t|\tau_0, t) = \mathcal{N}(\tau_t; \sqrt{\bar{\alpha}_t}\tau_0, (1-\bar{\alpha}_t)\mathbf{I})$, with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. This allows sampling τ_t without running the forward diffusion process [24].

The inverse (denoising) process transforms Gaussian noise back to the data distribution through a series of denoising steps $p(\tau_{t-1}|\tau_t, t)$. Diffusion models approximate this posterior distribution with a parametrized Gaussian $p_{\theta}(\tau_{t-1}|\tau_t, t) = \mathcal{N}(\tau_{t-1}; \mu_t = \mu_{\theta}(\tau_t, t), \Sigma_t)$.

For simplicity, only the mean of the inverse process is learned, and the covariance is set to $\Sigma_t = \sigma_t^2\mathbf{I} = \tilde{\beta}_t\mathbf{I}$, with $\tilde{\beta}_t = \beta_t(1 - \bar{\alpha}_{t-1})/(1 - \bar{\alpha}_t)$. [24] proposed that instead of learning the posterior mean directly, the noise ε can be learned instead, since $\mu_{\theta}(\tau_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\tau_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta}(\tau_t, t) \right)$, via a simplified loss function

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \varepsilon, \tau_0} [\|\varepsilon - \varepsilon_{\theta}(\tau_t, t)\|_2^2],$$

with $t \sim \mathcal{U}(1, N)$, $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, $\tau_0 \sim q(\tau_0)$, and $\tau_t = \sqrt{\bar{\alpha}_t}\tau_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$.

Following [38], we encode the diffusion model over trajectories with a temporal U-Net, which has proven to be a reasonable architecture for diffusion models over trajectories. Please consult [38] for details on the network architecture.

D. Optimal sampling with guidance

We describe how to directly sample from the posterior $p(\tau|\mathcal{O})$ using diffusion models, which is equivalent to sampling from the prior while biasing the trajectories towards the task likelihood. Note that τ results from the last step of the denoising process $\tau \equiv \tau_0$. By definition of the Markovian reverse diffusion $p(\tau_0|\mathcal{O}) = p(\tau_N|\mathcal{O}) \prod_{t=1}^N p(\tau_{t-1}|\tau_t, t, \mathcal{O})$, where $p(\tau_N|\mathcal{O})$ is standard Gaussian noise by definition. Hence, to sample from $p(\tau_0|\mathcal{O})$, we iteratively sample from the task-conditioned posterior

$$p(\tau_{t-1}|\tau_t, t, \mathcal{O}) \propto p(\tau_{t-1}|\tau_t, t)p(\mathcal{O}|\tau_{t-1}), \quad (4)$$

where $p(\mathcal{O}|\tau_{t-1}) = p(\mathcal{O}|\tau_t, \tau_{t-1}, t)$, i.e. we drop the conditioning on τ_t and t because the task is only conditioned on the current sample τ_{t-1} , and $p_{\theta}(\tau_{t-1}|\tau_t, t)$ is modeled with a diffusion model parametrized by θ .

To sample from the task-conditioned posterior $p(\tau_{t-1}|\tau_t, t, \mathcal{O})$, we will use a similar technique as in classifier guidance, since the posterior cannot be sampled in closed form [23], [49]. Considering the learned denoising prior model over trajectories is Gaussian, its logarithm evaluates to

$$\begin{aligned} \log p_{\theta}(\tau_{t-1}|\tau_t, t) &= \log \mathcal{N}(\tau_t; \mu_t = \mu_{\theta}(\tau_t, t), \Sigma_t) \\ &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t)^{\top} \Sigma_t^{-1}(\tau_{t-1} - \mu_t). \end{aligned} \quad (5)$$

By definition of the noise schedule β_t , as the denoising step approaches zero, so does the noise covariance $\lim_{t \rightarrow 0} \|\Sigma_t\| = 0$. Therefore, $p_{\theta}(\tau_{t-1}|\tau_t, t)$ concentrates all the mass close to the mean μ_t , and therefore the task log-likelihood is approximated with a first-order Taylor expansion around μ_t

$$\log p(\mathcal{O}|\tau_{t-1}) \approx \log p(\mathcal{O}|\tau_{t-1} = \mu_t) + (\tau_{t-1} - \mu_t)g, \quad (6)$$

with $g = \nabla_{\tau_{t-1}} \log p(\mathcal{O}|\tau_{t-1})|_{\tau_{t-1}=\mu_t}$. Combining (5) and (6) we obtain

$$\begin{aligned} \log p(\tau_{t-1}|\tau_t, t, \mathcal{O}) &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t)^{\top} \Sigma_t^{-1}(\tau_{t-1} - \mu_t) + (\tau_{t-1} - \mu_t)g \\ &\propto -\frac{1}{2}(\tau_{t-1} - \mu_t - \Sigma_t g)^{\top} \Sigma_t^{-1}(\tau_{t-1} - \mu_t - \Sigma_t g) \\ &= \log p(z), \text{ with } p(z) = \mathcal{N}(z; \mu_t + \Sigma_t g, \Sigma_t). \end{aligned}$$

Algorithm 1: Motion Planning Diffusion

TRAINING

Input: Collision-free trajectories $\mathcal{D}$, Diffusion model ε_θ , learning rate α , noise schedule terms $\bar{\alpha}_t$

```
1 while training is not finished do
  ▷ sample a batch of trajectories
2   $\tau_0 \sim \mathcal{D}$ ,  $\varepsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ,  $t \sim \mathcal{U}(1, N)$ 
  ▷ compute the diffusion loss function
3   $\tau_t = \sqrt{\bar{\alpha}_t} \tau_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ 
4   $\mathcal{L}(\theta) = \|\varepsilon - \varepsilon_\theta(\tau_t, t)\|_2^2$ 
  ▷ gradient update
5   $\theta = \theta + \alpha \nabla_\theta \mathcal{L}(\theta)$ 
```

INFERENCE

Input: Pre-trained diffusion model ε_θ , start and goal states (s_s, s_g) , motion planning costs c_i , temperatures λ_i , scheduling terms $(\alpha_t, \bar{\alpha}_t, \sigma_t)$

```
▷ sample a batch of trajectories
6  $\tau_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
  ▷ hard set start and goal states
7  $\tau_N[0] = s_s$ ,  $\tau_N[H-1] = s_g$ 
8 for  $t = N, \dots, 1$  do
  ▷ compute the diffusion prior mean
9   $\mu_t = \frac{1}{\sqrt{\alpha_t}} \left( \tau_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \varepsilon_\theta(\tau_t, t) \right)$ 
  ▷ compute weighted gradient of costs
10  $g = -\sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1} = \mu_t)$ 
  ▷ move trajectories to low cost regions
11  $\tau_{t-1} = \mu_t + g + \sigma_t z$ ,  $z \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
12  $\tau_{t-1}[0] = s_s$ ,  $\tau_{t-1}[H-1] = s_g$ 
```

Output: optimized batch of trajectories τ_0

Hence, sampling from the task-conditioned posterior is equivalent to sampling from a Gaussian distribution with mean and covariance as $p(z)$. In the motion planning-inference case, we have from (2)

$$\begin{aligned} g &= \nabla_{\tau_{t-1}} \log p(\mathcal{O} | \tau_{t-1})|_{\tau_{t-1}=\mu_t} \\ &= -\sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1})|_{\tau_{t-1}=\mu_t}, \end{aligned}$$

where the costs are differentiable w.r.t. the trajectory, e.g., smoothness or differentiable signed distance functions.

There are direct benefits to this formulation. While smoothly sampling trajectories from the diffusion prior, we can bias the samples towards regions that increase the task likelihoods, thus resulting in decreasing overall cost objective in (3). In motion planning, these likelihoods can be collision-free regions, goal sets, and joint limits of the configuration space. In practice, to keep the influence of the task likelihood, we typically drop the covariance Σ_t scaling (cf. line 11 in Algorithm 1), since it approaches zero when t approaches 0. This choice is equivalent to scaling $\lambda_i = \tilde{\lambda}_i / \sigma_t$ with time-dependent variances σ_t , where λ_i are the constant hyperparameters, which can be interpreted as increasing likelihood relevancy when $\sigma_t \rightarrow 0$. Algorithm 1 depicts the pseudo-code for training and inference of diffusion models for motion planning.

E. Motion planning costs

We briefly describe the standard motion planning costs used in the objective (3). Note that for cost functions defined on waypoints, the total cost for a trajectory is the sum of waypoint costs, i.e. $c(\tau) = \sum_i \sum_{j=0}^{H-1} c_i(\tau[j])$.

Start and Goal States Cost. For the diffusion model, the start (s_s) and goal (s_g) state distribution is represented as a Dirac delta, i.e., one factor of (2) for the start state equates to $p(\tau[0] = s_s | \tau) = \delta_{\tau[0]=s_s}(\tau)$. In practice, this is implemented by hard setting the initial and final configurations of the trajectory as in lines 7 and 12 of Algorithm 1 [38]. If the generative model does not support Dirac delta distributions, this cost can also be formulated as a quadratic cost, e.g. for the start state $c_{s_s}(\tau) = \|s_s - \tau[0]\|_2^2$.

Collision Cost. Similar to GPMP [11], we populate K collision spheres on the robot body. Given differentiable forward kinematics implemented in PyTorch [50], with the kinematics Jacobian computed by auto-differentiation, the obstacle cost for any configuration q is $c_{\text{obs}}(q) = \frac{1}{K} \sum_{j=1}^K c(x(q, S_j))$ with $x(q, S_j)$ is the forward kinematics position of the j^{th} -collision sphere. For gradient-based motion optimizers, we design the cost using the differentiable signed-distance function $d(\cdot)$ from the sphere center to the closest obstacle surface (minus the sphere radius) in the task space with a

$$\epsilon > 0 \text{ margin } c(x) = \begin{cases} -d(x) + \epsilon & \text{if } d(x) \leq \epsilon \\ 0 & \text{if } d(x) > \epsilon \end{cases}.$$

Self-collision Cost. We group the collision spheres that belong to the same robot links. Then, we compute the pair-wise link sphere distances. The self-collision cost is the average of the computed pair-wise distances.

Joint Limits Cost. We enforce joint limits (and velocity limits) by computing the L2 norm joint violations as costs, with a $\epsilon > 0$ margin on each dimension i

$$c_{\text{limits}}(q_i) = \begin{cases} \|q_{\min} + \epsilon - q_i\|_2^2 & \text{if } q_i < q_{\min} + \epsilon \\ 0 & \text{if } q_{\min} + \epsilon \leq q_i \leq q_{\max} - \epsilon \\ \|q_{\max} - \epsilon - q_i\|_2^2 & \text{else} \end{cases}.$$

End-effector Cost. The distance on SE(3) state space defines the end-effector cost. Given two points $T_1 = [R_1, p_1]$ and $T_2 = [R_2, p_2]$ in SE(3), consisting of a translational and rotational part, we choose the following distance on SE(3) $d_{\text{SE}(3)}(T_1, T_2) = \|p_1 - p_2\|_2^2 + \|\text{LogMap}(R_1^T R_2)\|$, where $\text{LogMap}(\cdot)$ is the operator that maps an element of SO(3) to its tangent space [51]. Then, given the current end-effector pose $T(q)$ computed via forward kinematics and the goal pose T_g , the end-effector cost is

$$c_{ee}(q) = d_{\text{SE}(3)}(T(q), T_g). \quad (7)$$

Gaussian Process Cost. The uncontrolled trajectory distribution can be represented as the zero-mean GP prior [52], [11] $q_F(\tau) = \mathcal{N}(\tau; \mathbf{0}, \mathbf{K})$, with a constant time discretization Δt , and time-correlated covariance matrix $\mathbf{K} = [\mathbf{K}(i, j)]_{i,j,0 \leq i,j \leq H-1}$, $\mathbf{K}(i, j) \in \mathbb{R}^{d \times d}$. We can factorize the GP prior by following [52], i.e.,

$$q_F(\tau) \propto \exp\left(-\frac{1}{2}\|\tau\|_{\mathbf{K}^{-1}}^2\right) \propto \prod_{t=0}^{H-1} q_t(s_t, s_{t+1}),$$

with each binary GP factor defined as

$$q_t(\mathbf{s}_t, \mathbf{s}_{t+1}) \propto \exp \left\{ -\frac{1}{2} \|\Phi_{t,t+1} \mathbf{s}_t - \mathbf{s}_{t+1}\|_{\mathbf{Q}_{t,t+1}^{-1}}^2 \right\},$$

with $\Phi_{t,t+1}$ the state transition matrix, and $\mathbf{Q}_{t,t+1}$ the covariance between time steps t and $t+1$. In this work, we assume a holonomic system model and the state transition and covariance matrices are

$$\Phi_{t,t+1} = \begin{bmatrix} \mathbf{I} & \Delta t \mathbf{I} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \quad \mathbf{Q}_{t,t+1} = \begin{bmatrix} \frac{1}{3} \Delta t^3 \mathbf{Q}_c & \frac{1}{2} \Delta t^2 \mathbf{Q}_c \\ \frac{1}{2} \Delta t^2 \mathbf{Q}_c & \Delta t \mathbf{Q}_c \end{bmatrix}$$

respectively, with $\mathbf{Q}_c$ the power-spectral density matrix. Via the planning-as-inference perspective, the GP cost is

$$c_{\text{GP}}(\tau) = -\log q_F(\tau) = \frac{1}{2} \sum_{t=0}^H \|\Phi_{t,t+1} \mathbf{s}_t - \mathbf{s}_{t+1}\|_{\mathbf{Q}_{t,t+1}^{-1}}^2.$$

In essence, this cost promotes dynamic feasibility and smoothness. More details on GP priors can be found in [52].

Note that we can introduce further costs, such as manipulability, depending on task requirements.

IV. EXPERIMENTS

To verify the advantages of our approach, we perform experiments to answer the following questions: (1) Can the diffusion model learn collision-free high-dimensional and highly-multimodal trajectory distributions? (2) How does the diffusion model prior compares to a commonly used Conditional Variational AutoEncoder (CVAE) as a trajectory generative model? (3) Can our approach generate collision-free trajectories in the presence of obstacles not seen during training? (4) Does merging sampling from the prior while biasing samples with cost gradients improve results, compared to other generative models that first sample from the prior and then optimize the likelihood? (5) Is the diffusion model a good prior for optimization-based planning algorithms?

A. Experimental Setup

Environments. We consider different environments reflecting increasing difficulty, as depicted in Fig. 2. PointMass2D Dense is a 2D planar navigation environment with randomly placed spheres and boxes that a point mass robot has to navigate. PointMass3D Maze Boxes is a 3D maze environment with boxes and narrow passages. Panda Spheres has a 7-dof Franka Emika Panda robot and uniformly random spheres in its workspace, which benchmarks planners in higher-dimensional and non-trivial collision-free configuration space manifold. Panda Shelf has a 7-dof Panda robot placed on a working bench and with a nearby shelf, which replicates realistic tasks as a proof-of-concept of our method. The trajectory generative models (diffusion and CVAE) are trained with expert data generated from these environments. To test the generalization capabilities of the learning-based methods, all environments are extended with additional sphere and box obstacles that are not present during training - identified with the suffix “Extra Obstacles”.

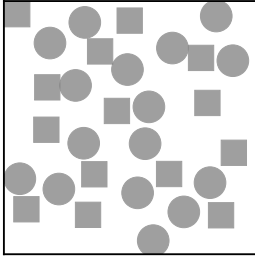
Tasks. In all environments, the task is to find a collision-free and smooth trajectory starting and finishing at random configuration-space positions $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$, which do not

result in a task-space collision. In the real-world Panda Shelf environment, besides finding a collision-free path, we additionally add an extra cost to maintain the end-effector orientation constant along the trajectory. Using (7), the trajectory cost is $c_{\text{EE}}(\tau) = \sum_{k=0}^{H-1} d_{\text{SE}(3)}(\mathbf{T}(\mathbf{q}[k]), \mathbf{T}_g[k])$, where $\mathbf{T}(\mathbf{q}[k])$ is the end-effector pose at waypoint k and $\mathbf{T}_g[k]$ the desired one. Since we only want to enforce the orientations, we only set the desired goal orientation and copy the desired end-effector position from the waypoint position, i.e., $\mathbf{T}(\mathbf{q}[k]) = [\mathbf{R}_k, \mathbf{p}_k]$, $\mathbf{T}_g[k] = [\mathbf{R}_g, \mathbf{p}_k]$.

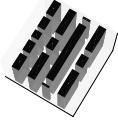
Algorithms and Baselines. We denote the diffusion model trained over collision-free trajectories as DIFFUSIONPRIOR, and our proposed approach of sampling with cost guidance (sec. III-D) as MPD. These methods are compared against a set of baselines to showcase different properties. A simple baseline is a sample-based method RRTConnect [8]. We modify this algorithm in several ways to fully use the GPU parallelization capabilities: at the start of optimization, we pre-compute a buffer of collision-free configurations (the computation time of this operation is negligible due to parallelization); a sample is removed from the buffer when added to the search tree; the buffer is refilled when there are no samples left; and computing the nearest neighbor node is done in parallel using the GPU. These improvements lead to very fast planning for RRTConnect.

On the choice of the optimization-based planner, we use GPMP (i.e., GPMP2 in [11]) without an informative prior - a constant-velocity straight line connecting the start and goal configurations. On conditional trajectory generative model as a baseline, we learn a Conditional Variational AutoEncoder (CVAE) [53], [15], denoted as CVAEPrior. For a fair comparison, the encoder and decoder networks are the same as the encoding and decoding parts of the U-Net diffusion model. The conditioning variable is the start and goal configurations. These are stacked with the learned trajectory representation of the encoder and then passed through a neural network to encode a Gaussian posterior in the latent space. We ran a hyperparameter search on the latent space dimension, learning rate, and the KL regularization multiplier and chose the model with the best validation loss on the same dataset used to train the diffusion model. We analyze sampling from this prior first and afterward optimize the trajectory cost likelihood - this method is named CVAEPosterior. The number of optimization steps is equivalent to the one MPD uses. We use the same cost temperatures λ_i for both MPD and CVAEPosterior. Finally, the learned diffusion and CVAE models can be used as a prior for GPMP, as well as a simple RRTConnect prior, which can be a way to provide an initial trajectory for optimization-based motion planning [2]. These are referred to as [Prior]-GPMP. If RRTConnect is used as a prior, we smoothen that solution using B-splines before passing it to GPMP.

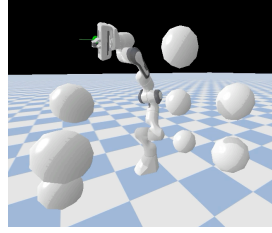
Metrics. A set of commonly used metrics for planning evaluation are used. For all metrics, we report the mean and standard deviation of motion planning results of 100 random contexts (start and goal configuration-free positions) when sampling 100 trajectories per context. Note that for algorithms that use RRTConnect, the trajectories for each



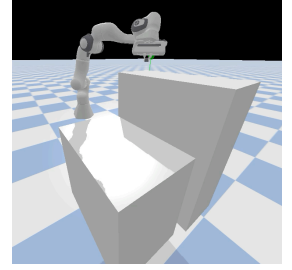
(a) PointMass2D Dense



(b) PointMass3D Maze Boxes



(c) Panda Spheres



(d) Panda Shelf

Fig. 2: The environments considered in our experiments include robot navigation tasks of a point mass in 2D and 3D, and a 7-dof Franka Emika Panda manipulator. In (a) and (b), the green and red dots are the initial and goal states, respectively, and the blue line is the result of RRT Connect.

TABLE I: Motion planning generation benchmarks in the environments of the training set, and additional environments with extra obstacles. $\uparrow$ means higher is better. $\downarrow$ means lower is better. Values highlighted in bold are discussed in the main text. Legend: RRTC – RRTConnect, CPrior – CVAE Prior, DPrior – Diffusion Prior, CPost – CVAE Posterior Optimization, MPD – Motion Planning Diffusion

	PointMass2D Dense					PointMass2D Dense - Extra Obstacles				
	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$
RRTC	4.7 $\pm$ 2.5	100.0 $\pm$.0	.0 $\pm$.0	1.9 $\pm$.5		8.0 $\pm$ 3.5	100.0 $\pm$.0	.0 $\pm$.0	2.2 $\pm$.6	
CPrior	.02 $\pm$.2	46.0 $\pm$ 49.5	11.8 $\pm$ 12.5	1.5 $\pm$.4	.0 $\pm$.0	.02 $\pm$.2	14.0 $\pm$ 34.7	24.3 $\pm$ 14.7	1.6 $\pm$.4	.0 $\pm$.0
DPrior	.3 $\pm$.0	98.0 $\pm$ 14.0	5.5 $\pm$ 06.2	1.6 $\pm$.5	1.3 $\pm$ 1.0	.3 $\pm$.0	60.0 $\pm$ 49.0	16.0 $\pm$ 10.5	1.5 $\pm$.5	1.3 $\pm$ 1.0
GPMP	26.5 $\pm$.2	52.0 $\pm$ 49.7	1.7 $\pm$ 1.3	1.4 $\pm$.4	.03 $\pm$.03	26.5 $\pm$.2	49.0 $\pm$ 50.0	1.1 $\pm$ 1.5	1.4 $\pm$.4	.03 $\pm$.04
RRTC-GPMP	38.43 $\pm$ 30.9	100.0 $\pm$.0	.0 $\pm$ 0.0	1.94 $\pm$.5	2.2 $\pm$.9	42.5 $\pm$ 7.2	100.0 $\pm$.0	0.0 $\pm$ 0.1	2.0 $\pm$.5	2.7 $\pm$.9
CPrior-GPMP	25.3 $\pm$.2	71.0 $\pm$ 45.4	0.9 $\pm$ 01.8	1.4 $\pm$.5	.01 $\pm$.02	26.0 $\pm$.2	53.0 $\pm$ 49.9	01.1 $\pm$ 01.7	1.5 $\pm$.5	.02 $\pm$.03
DPrior-GPMP	26.0 $\pm$.3	99.0 $\pm$ 10.0	.3 $\pm$.6	1.6 $\pm$.5	1.4 $\pm$ 1.1	27.0 $\pm$.3	92.0 $\pm$ 27.1	.7 $\pm$.8	1.6 $\pm$.5	1.4 $\pm$.9
CPost	.1 $\pm$.1	88.0 $\pm$ 32.5	.3 $\pm$.7	1.5 $\pm$.4	.01 $\pm$.02	.1 $\pm$.1	78.0 $\pm$ 41.4	.5 $\pm$.7	1.62 $\pm$.5	.01 $\pm$.02
MPD	.3 $\pm$.0	99.0 $\pm$ 10.0	.6 $\pm$ 1.2	1.7 $\pm$.5	1.4 $\pm$ 1.0	.3 $\pm$.0	79.0 $\pm$ 40.7	10.3 $\pm$ 8.5	1.7 $\pm$.4	1.4 $\pm$ 1.0

	PointMass3D Maze Boxes					PointMass3D Maze Boxes - Extra Obstacles				
	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$
RRTC	27.4 $\pm$ 26.1	100.0 $\pm$.0	.0 $\pm$.0	3.7 $\pm$ 1.8		32.0 $\pm$ 27.4	100.0 $\pm$.0	.0 $\pm$.0	4.0 $\pm$ 1.7	
CPrior	.02 $\pm$.1	8.0 $\pm$ 27.1	29.1 $\pm$ 15.1	2.3 $\pm$.8	.01 $\pm$.02	.02 $\pm$.1	4.0 $\pm$ 19.6	27.9 $\pm$ 14.0	2.2 $\pm$.8	.01 $\pm$.02
DPrior	.3 $\pm$.0	54.0 $\pm$ 49.8	15.5 $\pm$ 11.7	2.1 $\pm$.7	4.1 $\pm$ 3.9	.3 $\pm$.0	51.0 $\pm$ 50.0	14.8 $\pm$ 8.4	2.2 $\pm$.8	4.1 $\pm$ 4.1
GPMP	47.4 $\pm$.1	16.0 $\pm$ 36.7	32.3 $\pm$ 16.6	1.5 $\pm$.5	.01 $\pm$.0	48.5 $\pm$.1	19.0 $\pm$ 39.2	31.2 $\pm$ 17.9	1.4 $\pm$.4	.01 $\pm$.0
RRTC-GPMP	114.6 $\pm$ 56.9	100.0 $\pm$.0	2.2 $\pm$ 1.3	4.1 $\pm$ 1.6	5.5 $\pm$ 3.0	111.2 $\pm$ 52.4	100.0 $\pm$.0	2.4 $\pm$ 1.1	3.9 $\pm$ 1.4	5.59 $\pm$ 3.1
CPrior-GPMP	47.5 $\pm$.3	8.0 $\pm$ 27.1	27.7 $\pm$ 15.3	2.2 $\pm$.8	.01 $\pm$.03	47.9 $\pm$.3	11.0 $\pm$ 31.3	29.0 $\pm$ 16.3	2.1 $\pm$.7	.01 $\pm$.03
DPrior-GPMP	48.3 $\pm$.2	39.0 $\pm$ 48.8	14.4 $\pm$ 7.7	2.2 $\pm$.8	4.3 $\pm$ 4.1	49.0 $\pm$.2	43.0 $\pm$ 49.5	15.1 $\pm$ 8.5	2.1 $\pm$.8	4.2 $\pm$ 3.9
CPost	.1 $\pm$.2	50.0 $\pm$ 50.0	01.4 $\pm$ 01.4	2.3 $\pm$.7	.02 $\pm$.03	.1 $\pm$.2	52.0 $\pm$ 50.0	01.3 $\pm$ 01.5	2.2 $\pm$.7	.02 $\pm$.03
MPD	.3 $\pm$.01	85.0 $\pm$ 35.7	2.0 $\pm$ 1.8	2.4 $\pm$.7	4.2 $\pm$ 4.0	.3 $\pm$.01	82.0 $\pm$ 38.4	3.1 $\pm$ 2.1	2.4 $\pm$.8	4.2 $\pm$ 4.1

	Panda Spheres					Panda Spheres - Extra Obstacles				
	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$
RRTC	42.9 $\pm$ 18.8	100.0 $\pm$.0	.04 $\pm$.08	12.1 $\pm$ 2.8		90.2 $\pm$ 103.6	100.0 $\pm$.0	0.05 $\pm$ 0.05	13.2 $\pm$ 3.8	
CPrior	.02 $\pm$.1	36.0 $\pm$ 48.0	14.9 $\pm$ 15.0	5.4 $\pm$ 1.0	.02 $\pm$.00	.02 $\pm$.1	10.0 $\pm$ 30.0	35.2 $\pm$ 24.6	5.4 $\pm$ 1.2	.01 $\pm$.01
DPrior	.3 $\pm$.0	88.0 $\pm$ 32.5	11.5 $\pm$ 6.0	7.8 $\pm$ 1.4	17.3 $\pm$ 5.0	.3 $\pm$.0	78.0 $\pm$ 41.4	23.9 $\pm$ 7.4	7.5 $\pm$ 1.6	18.0 $\pm$ 4.2
GPMP	194.4 $\pm$.1	42.0 $\pm$ 49.4	4.1 $\pm$ 4.8	5.1 $\pm$ 1.2	.02 $\pm$.05	194.5 $\pm$.2	28.0 $\pm$ 44.9	9.6 $\pm$ 8.2	5.1 $\pm$ 1.3	.03 $\pm$.12
RRTC-GPMP	230.4 $\pm$ 14.4	100.0 $\pm$.0	02.4 $\pm$ 02.7	7.9 $\pm$ 1.4	18.7 $\pm$ 5.6	253.6 $\pm$ 58.7	96.0 $\pm$ 19.6	4.1 $\pm$ 2.8	8.0 $\pm$ 1.4	20.6 $\pm$ 7.2
CPrior-GPMP	191.6 $\pm$.3	34.0 $\pm$ 47.4	5.2 $\pm$ 5.7	5.1 $\pm$.9	.03 $\pm$.06	194.2 $\pm$.1	18.0 $\pm$ 38.4	12.3 $\pm$ 10.3	5.1 $\pm$ 1.1	.04 $\pm$.06
DPrior-GPMP	192.3 $\pm$.1	100.0 $\pm$.0	4.0 $\pm$ 2.6	6.8 $\pm$ 1.2	15.6 $\pm$ 3.5	192.1 $\pm$.1	82.0 $\pm$ 38.4	7.8 $\pm$ 3.8	7.1 $\pm$ 1.0	16.0 $\pm$ 3.6
CPost	.8 $\pm$.1	70.0 $\pm$ 45.8	.7 $\pm$ 1.0	8.0 $\pm$ 1.3	.03 $\pm$.04	.8 $\pm$.1	45.0 $\pm$ 49.8	1.5 $\pm$ 1.7	8.0 $\pm$ 1.2	.1 $\pm$.1
MPD	1.1 $\pm$.01	100.0 $\pm$.0	1.2 $\pm$.8	9.9 $\pm$ 1.2	17.4 $\pm$ 3.9	1.1 $\pm$.01	93.0 $\pm$ 25.5	12.5 $\pm$ 7.6	10.0 $\pm$ 1.2	18.0 $\pm$ 4.0

	Panda Shelf					Panda Shelf - Extra Obstacles				
	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$	T[s] $\downarrow$	S[%] $\uparrow$	I[%] $\downarrow$	PL $\downarrow$	VAR $\uparrow$
RRTC	29.9 $\pm$ 8.3	100.0 $\pm$.0	0.04 $\pm$ 0.11	11.2 $\pm$ 2.4		31.5 $\pm$ 9.4	99.0 $\pm$ 10.0	2.2 $\pm$ 4.5	11.5 $\pm$ 2.9	
CPrior	.03 $\pm$.2	92.0 $\pm$ 27.1	1.8 $\pm$ 6.5	4.8 $\pm$.9	.02 $\pm$.0	.03 $\pm$.1	68.0 $\pm$ 46.7	8.3 $\pm$ 14.1	4.6 $\pm$.8	.01 $\pm$.00
DPrior	.3 $\pm$.0	100.0 $\pm$.0	3.6 $\pm$ 4.8	7.6 $\pm$ 1.2	14.5 $\pm$ 3.4	.3 $\pm$.0	100.0 $\pm$.0	5.9 $\pm$ 7.3	7.6 $\pm$ 1.3	14.3 $\pm$ 3.7
GPMP	192.1 $\pm$.1	88.0 $\pm$ 32.5	.5 $\pm$ 1.6	5.1 $\pm$ 1.3	.01 $\pm$.02	193.1 $\pm$.13	82.0 $\pm$ 38.42	2.0 $\pm$ 5.1	5.1 $\pm$ 1.4	.01 $\pm$.01
RRTC-GPMP	218.3 $\pm$ 7.0	100.0 $\pm$.0	.4 $\pm$.9	8.1 $\pm$ 1.1	16.7 $\pm$ 3.8	228.2 $\pm$ 20.1	98.0 $\pm$ 14.0	1.5 $\pm$ 2.1	7.9 $\pm$ 1.5	18.3 $\pm$ 6.0
CPrior-GPMP	192.9 $\pm$.1	94.0 $\pm$ 23.8	.6 $\pm$ 2.6	4.4 $\pm$.9	.01 $\pm$.00	194.3 $\pm$.2	60.0 $\pm$ 49.0	3.3 $\pm$ 5.7	4.5 $\pm$.8	.03 $\pm$.04
DPrior-GPMP	192.3 $\pm$.1	100.0 $\pm$.0	0.9 $\pm$ 1.1	7.1 $\pm$ 1.1	13.8 $\pm$ 2.9	192.6 $\pm$.1	98.0 $\pm$ 14.0	2.3 $\pm$ 2.4	6.9 $\pm$ 1.2	14.0 $\pm$ 3.1
CPost	.8 $\pm$.1	93.0 $\pm$ 25.5	0.1 $\pm$ 0.4	7.4 $\pm$ 1.2	.02 $\pm$.04	.8 $\pm$.1	83.0 $\pm$ 37.6	0.5 $\pm$ 1.1	10.8 $\pm$ 6.8	2.4 $\pm$ 5.2
MPD	1.0 $\pm$.01	100.0 $\pm$.0	0.6 $\pm$ 0.9	9.3 $\pm$ 1.0	14.8 $\pm$ 3.2	1.0 $\pm$.01	99.0 $\pm$ 10.0	4.1 $\pm$ 4.9	9.4 $\pm$ 1.1	15.1 $\pm$ 3.5

context are sampled sequentially since GPU batch parallelization is not trivial for these methods. To assess the speed of RRTConnect in generating a single collision-free path, the reported times can be divided by 100, making the algorithm, on average, faster than one second. Time (T) - is the planner's computational time to generate/optimize the required number of trajectories. Success (S) - is the success rate, which is 1 if at least one of the trajectories in the batch is collision-free, and 0 otherwise. Intensity (I) - is the percentage of the

waypoints that are in collision, which assesses the ability to generate almost-collision-free trajectories. Path Length (PL) - is the trajectory length. Waypoint variance (VAR) - is the sum (along the trajectory dimension) of the pairwise L2-distance variance between waypoints at corresponding time steps. This metric measures how multimodal (spread) the generated trajectories are.

Dataset generation and training. To generate multimodal and collision-free expert data trajectories in each environ-

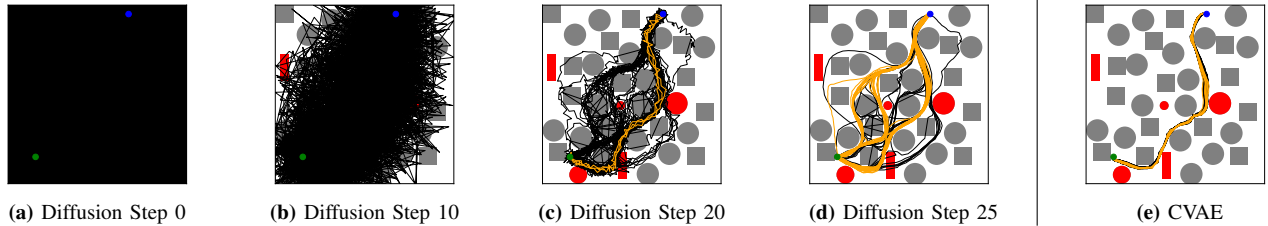


Fig. 3: (a)-(d) Diffusion steps of MPD on a batch of 100 trajectories in the PointMass2D Dense - Extra Obstacles environment. Notice how noise transforms into multimodal, smooth and collision-free trajectories. (e) Trajectories generated by CVAEPosterior. Obstacles in red were not present in the training environment. Trajectories in orange are collision-free, black in collision. The start and goal configurations are in green and blue.

ment, we sample 500 random start and goal context configurations and 20 trajectories per context. We use RRTConnect to get a rough initial solution, then smoothen it using a B-spline, and run many optimization steps of Stochastic-GPMP [13] to create collision-free and smooth trajectories, similar as [54]. This process is costly but done offline once. The data is split into training and validation datasets (5% of the data). The input to the diffusion and CVAE models is a trajectory of dimension $H \times d$, with H the horizon and d the state-space dimension, e.g., we used $H = 64$ and $d = 14$ in the panda environments. The horizon value is a hyperparameter, but 64 was sufficient for the motion planning tasks we considered. The diffusion models are trained for 25 diffusion steps with exponential scheduling (we found it to work better than linear scheduling [24]). The models are trained using early stopping by inspecting the validation loss.

Implementation. For maximal parallelization, all environments, algorithms, and costs were implemented in PyTorch [50] utilizing the GPU. The experiments were conducted on a machine with an AMD EPYC 7453 28-Core Processor and NVIDIA GeForce RTX 3090.

B. Results in Simulation

The summarized results from planning in simulated environments are detailed in Table I. To answer our first and second questions, we look into the environments from the training set and observe that DiffusionPrior can generate more collision-free trajectories than the baseline CVAEPrior. E.g., in the PointMass2D Dense environment, the success rate is 98% vs. 46%. Moreover, the diffusion model can produce more multimodal trajectories, as the variance column shows. Empirically, we observed that across different planning problems, the CVAE models tended to generate fewer modes than the ones from the diffusion.

To assess questions 3 and 4, we look into the environments with the suffix “Extra Obstacles”, where new obstacles not seen during training are placed randomly in the environment. First, we observe that the success rate by sampling only from the priors decreases, which is expected since they were not trained in these environments. Note that in the Panda Shelf environment, the success rate of DiffusionPrior does not decrease from 100%, but the collision intensity increases from 3.6% to 5.9%. Observing the success rates of MPD and comparing them to CVAEPosterior, we see that using the diffusion model in combination with the cost gradients yields better results in terms of success rate and multimodality (as the variance measures the spread of the distribution). This phenomenon can be seen in Fig. 3, where we compare

the optimization with the diffusion and CVAE models. A common criticism of diffusion models is their sampling time. However, for motion planning, the computation times of both methods for the Panda environments are comparable since most of the cost is spent in the gradient computation of the cost and not in the diffusion sampling.

Finally, we check if the diffusion samples act as a good prior for GPMP. Across all environments, using the model samples as initialization for GPMP generated higher success rates and multimodal trajectories compared to running GPMP with a constant velocity straight-line mean prior. The low success rate of GPMP is more noticeable in environments where the collision-free space manifold is very complex, as in the PointMass2D Dense environment. The larger compute times of algorithms using GPMP are due to running this algorithm with a large batch of trajectories since at every time-step it solves a trust region gradient.

The sampling times for RRTConnect include sequentially sampling trajectories using the GPU since, as noted before, parallelization of RRT-like algorithms for a batch is not trivial. Even though RRTConnect also is a fast prior for GPMP, it generally produces high-jerk trajectories with higher path lengths, which would need more optimization steps of GPMP to achieve the same level of the DiffusionPrior.

C. Results in the Real World Panda Shelf

We test our method in a real-world task where a robot moves a bottle while avoiding obstacles not present in the training set and maintaining a constant end-effector orientation. We approximate the new obstacles (cf. Fig. 1) as box models. With the robot in gravity compensation mode, we record 3 different initial configurations and one desired goal configuration. For each start-goal pair, we sampled 10 trajectories with the MPD, filtered the ones in collision, obtaining 25 collision-free trajectories, and without selecting them with respect to any criteria, ran them in the real robot using a joint impedance controller. We obtained 19/25 collision-free trajectories, demonstrating our method’s efficacy of generating diverse success solutions. We hypothesize that the remaining 6 trajectories that collided with the new obstacles are due to the approximated robot (spheres) collision model.

V. CONCLUSION AND FUTURE WORK

In this paper, we proposed using diffusion models as priors for bootstrapping motion planning problems, via the planning-as-inference perspective. We parameterize a trajectory with waypoints and construct a generative model over the whole trajectory. We train this model via supervised

learning on motion plans generated with an optimal planner. At inference time, instead of sampling trajectories from the prior to only initialize an optimization-based motion planner, we propose to use the guidance properties of diffusion models to concurrently sample from the prior and bias these samples towards regions of low cost (high likelihood).

Our results show several benefits of diffusion models. Due to their modeling capabilities, diffusion models are better priors for motion planning because they can encode multimodal trajectories better than commonly used CVAEs. Sampling from the learned prior while optimizing the likelihood leads to improved results in finding collision-free trajectories. Future work will include continuing to extend the usage of diffusion models to encode different parametrizations of trajectories for robotic movements.

REFERENCES

- [1] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge University Press, 2006.
- [2] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “Chomp: Gradient optimization techniques for efficient motion planning,” in *IEEE International Conference on Robotics and Automation*, 2009.
- [3] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, “Stomp: Stochastic trajectory optimization for motion planning,” in *IEEE International Conference on Robotics and Automation*, 2011.
- [4] M. Elbanhawi and M. Simic, “Sampling-based robot motion planning: A review,” *IEEE Access*, vol. 2, pp. 56–77, 2014.
- [5] K. Lynch and F. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [6] L. Kavraki, P. Svestka, J.-C. Latombe, and M. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, 1996.
- [7] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” 1998.
- [8] J. Kuffner and S. LaValle, “Rrt-connect: An efficient approach to single-query path planning,” in *IEEE ICRA*, 2000.
- [9] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *IJRR*, vol. 30, no. 7, pp. 846–894, 2011.
- [10] J. Dong, M. Mukadam, F. Dellaert, and B. Boots, “Motion planning as probabilistic inference using Gaussian processes and factor graphs,” in *RSS*, 2016.
- [11] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, “Continuous-time gaussian process motion planning via probabilistic inference,” *Int. J. Robotics Res.*, vol. 37, no. 11, 2018.
- [12] K. K. Hauser and V. Ng-Thow-Hing, “Fast smoothing of manipulator trajectories using optimal bounded-acceleration shortcuts,” in *ICRA*. IEEE, 2010, pp. 2493–2498.
- [13] J. Urain, A. Le, A. Lambert, G. Chalvatzaki, B. Boots, and J. Peters, “Learning implicit priors for motion optimization,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2022.
- [14] J. Wang *et al.*, “A survey of learning-based robot motion planning,” *IET Cyber-Systems and Robotics*, vol. 3, no. 4, pp. 302–314, 2021.
- [15] B. Ichter, J. Harrison, and M. Pavone, “Learning sampling distributions for robot motion planning,” in *IEEE ICRA*, 2018.
- [16] A. H. Qureshi, A. Simeonov, M. J. Bency, and M. C. Yip, “Motion planning networks,” in *IEEE ICRA*, 2019.
- [17] D. Koert, G. Maeda, R. Lioutikov, G. Neumann, and J. Peters, “Demonstration based trajectory optimization for generalizable robot motions,” in *IEEE-RAS Humanoids*, 2016, pp. 515–522.
- [18] M. A. Rana, M. Mukadam, S. R. Ahmadzadeh, S. Chernova, and B. Boots, “Towards robust skill generalization: Unifying learning from demonstration and motion planning,” in *CoRL*. PMLR, 2017.
- [19] A. T. Le *et al.*, “Learning forceful manipulation skills from multi-modal human demonstrations,” in *IEEE/RSJ IROS*, 2021.
- [20] J. Urain, N. Funk, J. Peters, and G. Chalvatzaki, “Se(3)-diffusionfields: Learning smooth cost functions for joint grasp and motion optimization through diffusion,” in *IEEE ICRA*, 2023.
- [21] J. Ortiz-Haro, J.-S. Ha, D. Driess, and M. Toussaint, “Structured deep generative models for sampling on constraint manifolds in sequential manipulation,” in *CoRL*. PMLR, 2022.
- [22] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, “An algorithmic perspective on imitation learning,” *Found. Trends Robotics*, vol. 7, no. 1-2, pp. 1–179, 2018.
- [23] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *ICML*. JMLR.org, 2015.
- [24] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *NeurIPS*. Curran Associates Inc., 2020.
- [25] Y. Song and S. Ermon, “Generative modeling by estimating gradients of the data distribution,” in *NeurIPS*, 2019.
- [26] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” 2021.
- [27] G. Kim, T. Kwon, and J. C. Ye, “Diffusionclip: Text-guided diffusion models for robust image manipulation,” in *IEEE/CVF CVPR*, 2022.
- [28] A. Ramesh *et al.*, “Hierarchical text-conditional image generation with clip latents,” 2022. [Online]. Available: <https://arxiv.org/abs/2204.06125>
- [29] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” 2021.
- [30] J. J. Johnson *et al.*, “Motion planning transformers: A motion planning framework for mobile robots,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.02791>
- [31] A. H. Qureshi and M. C. Yip, “Deeply informed neural sampling for robot motion planning,” in *IEEE/RSJ IROS*, 2018.
- [32] J. Wang, W. Chi, C. Li, C. Wang, and M. Q.-H. Meng, “Neural rrt*: Learning-based optimal path planning,” *IEEE T-ASE*, 2020.
- [33] W. Grathwohl, J. Kelly, M. Hashemi, M. Norouzi, K. Swersky, and D. Duvenaud, “No mcmc for me: Amortized sampling for fast and stable training of energy-based models,” 2020.
- [34] I. Kapelyukh, V. Vosylius, and E. Johns, “Dall-e-bot: Introducing web-scale diffusion models to robotics,” 2022.
- [35] W. Liu, T. Hermans, S. Chernova, and C. Paxton, “Strucdiffusion: Object-centric diffusion for semantic rearrangement of novel objects,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.04604>
- [36] E. J. C. Findlay, H. Zhang, Z. Chang, and H. P. H. Shum, “Denoising diffusion probabilistic models for styled walking synthesis,” 2022. [Online]. Available: <https://arxiv.org/abs/2209.14828>
- [37] J. Carvalho, M. Baierl, J. Urain, and J. Peters, “Conditioned score-based models for learning collision-free trajectory generation,” in *NeurIPS 2022 Workshop on Score-Based Methods*, 2022.
- [38] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *ICML*, 2022.
- [39] A. Ajay *et al.*, “Is conditional generative modeling all you need for decision-making?” 2022. [Online]. Available: <https://arxiv.org/abs/2211.15657>
- [40] H. Attias, “Planning by probabilistic inference,” in *Proceedings of the Ninth International Workshop on Artificial Intelligence and Statistics*, vol. R4. PMLR, 2003, pp. 9–16.
- [41] M. Toussaint, “Robot trajectory optimization using approximate inference,” in *ICML*. Association for Computing Machinery, 2009.
- [42] S. Levine, “Reinforcement learning and control as probabilistic inference: Tutorial and review,” 2018.
- [43] J. Peters, K. Mülling, and Y. Altun, “Relative entropy policy search,” in *AAAI*. AAAI Press, 2010.
- [44] J. Watson, H. Abdulsamad, and J. Peters, “Stochastic optimal control as approximate input inference,” in *CoRL*. PMLR, 2019.
- [45] J. Watson and J. Peters, “Inferring smooth control: Monte carlo posterior policy iteration with gaussian processes,” in *CoRL*, 2022.
- [46] A. T. Le, K. Hansel, J. Peters, and G. Chalvatzaki, “Hierarchical policy blending as optimal transport,” in *arXiv preprint arXiv:2212.01938*, 2022.
- [47] J. Dong, M. Mukadam, B. Boots, and F. Dellaert, “Sparse Gaussian processes on matrix Lie groups: A unified framework for optimizing continuous-time trajectories,” in *IEEE ICRA*, 2018.
- [48] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *ICML*. PMLR, 2021.
- [49] P. Dhariwal and A. Q. Nichol, “Diffusion models beat gans on image synthesis,” in *NeurIPS*, 2021, pp. 8780–8794.
- [50] A. Paszke *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *NeurIPS*. Curran Associates, Inc., 2019.
- [51] J. Sola, J. Deray, and D. Atchuthan, “A micro lie theory for state estimation in robotics,” *arXiv preprint arXiv:1812.01537*, 2018.
- [52] T. D. Barfoot, C. H. Tong, and S. Särkkä, “Batch continuous-time trajectory estimation as exactly sparse gaussian process regression,” in *RSS*, 2014.

- [53] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *NeurIPS*, vol. 28. Curran Associates, Inc., 2015.
- [54] M. Bhardwaj, B. Boots, and M. Mukadam, "Differentiable gaussian process motion planning," in *IEEE ICRA*, 2020.